

DozerNet — Team Setup & Handover Guide

You'll each receive a full, working copy of the project so it compiles and runs on your machine — but you only need to know your own module inside out for the viva. This guide shows: (A) the one-time setup everyone does, the same way, and (B) exactly which files are **yours** to understand and defend. Your team lead will send you the files described here.

#	Module	Owner	Package	Branch name
1	Customer Management	G. Ananian	module1_customer	feature/m1-customer-management
2	Booking & Rental Management	Theekshana A.D.	module2_booking	feature/m2-booking-rental
3	Fleet (JCB) Management	Jayawardhana A.G.D.L.	module3_fleet	feature/m3-fleet-management
4	Operator Management	Perera M.A.S.A.	module4_operator	feature/m4-operator-management
5	Maintenance & Service Management	Ruvendi R.G.K.	module5_maintenance	feature/m5-maintenance-service
6	Payment & Administration	Thathsara J.P.T.	module6_payment	feature/m6-payment-admin

How to use this guide: everyone reads Part A once (same steps for all six of you). Then jump to your own page in Part B for the exact file list that's your responsibility.

PART A — One-time setup (everyone does this)

A1. Install these first

- **JDK 21** (not newer — the test tooling doesn't support JDK 24+ yet). Check with `java -version`.
- **Maven 3.9+** — check with `mvn -version`.
- **MySQL 8** (Community Server + optionally MySQL Workbench).
- **Git**, and a GitHub account with a new empty repository ready for your module.
- An IDE (IntelliJ IDEA Community is fine) — open the project as a Maven project.

A2. Recreate this folder structure

Create these folders exactly as shown before copying in any files. `moduleN_xxx` is **your** module package from the table above.

```
dozernet/  
|-- pom.xml  
|-- README.md  
|-- .gitignore  
|-- src/  
| |-- main/  
| | |-- java/com/dozernet/  
| | | |-- DozerNetApplication.java  
| | | |-- DataSeeder.java  
| | | |-- RoleMigration.java  
| | | |-- common/ (shared - everyone gets this, see A3)  
| | | |-- moduleN_xxx/ (YOUR module - see Part B)  
| | | |-- entity/  
| | | |-- repository/  
| | | |-- service/  
| | | |-- web/  
| | | |-- dto/  
| | |-- resources/  
| | |-- application.properties  
| | |-- application-h2.properties  
| | |-- application-mysql.properties  
| | |-- static/ (css, js, images - shared)  
| | |-- templates/  
| | |-- fragments/, auth/, error/, notifications/, public/, admin/ (shared)  
| | |-- <your-template-folder>/ (YOUR module's pages)  
| |-- test/java/com/dozernet/...  
|-- docs/
```

A3. Files everyone receives (shared — needed to compile, don't edit without asking)

Root backend files

- `src/main/java/com/dozernet/DozerNetApplication.java`
- `src/main/java/com/dozernet/DataSeeder.java`
- `src/main/java/com/dozernet/RoleMigration.java`

common/ package (all 6 of you get every file in here)

- `src/main/java/com/dozernet/common/audit/AuditLog.java`
- `src/main/java/com/dozernet/common/audit/AuditLogRepository.java`

- src/main/java/com/dozernet/common/audit/AuditService.java
- src/main/java/com/dozernet/common/document/Document.java
- src/main/java/com/dozernet/common/document/DocumentRepository.java
- src/main/java/com/dozernet/common/document/DocumentService.java
- src/main/java/com/dozernet/common/document/DocumentType.java
- src/main/java/com/dozernet/common/exception/BusinessRuleException.java
- src/main/java/com/dozernet/common/exception/GlobalExceptionHandler.java
- src/main/java/com/dozernet/common/exception/ResourceNotFoundException.java
- src/main/java/com/dozernet/common/model/BaseEntity.java
- src/main/java/com/dozernet/common/model/Role.java
- src/main/java/com/dozernet/common/notification/EmailNotificationObserver.java
- src/main/java/com/dozernet/common/notification/InAppNotificationObserver.java
- src/main/java/com/dozernet/common/notification/Notification.java
- src/main/java/com/dozernet/common/notification/NotificationEvent.java
- src/main/java/com/dozernet/common/notification/NotificationObserver.java
- src/main/java/com/dozernet/common/notification/NotificationRepository.java
- src/main/java/com/dozernet/common/notification/NotificationService.java
- src/main/java/com/dozernet/common/security/CurrentUserService.java
- src/main/java/com/dozernet/common/security/CustomUserDetailsService.java
- src/main/java/com/dozernet/common/security/RoleRedirectSuccessHandler.java
- src/main/java/com/dozernet/common/security/SecurityConfig.java
- src/main/java/com/dozernet/common/user/AccountService.java
- src/main/java/com/dozernet/common/user/User.java
- src/main/java/com/dozernet/common/user/UserRepository.java
- src/main/java/com/dozernet/common/validation/ValidationPatterns.java
- src/main/java/com/dozernet/common/web/AuthController.java
- src/main/java/com/dozernet/common/web/DocumentController.java
- src/main/java/com/dozernet/common/web/GlobalModelAdvice.java
- src/main/java/com/dozernet/common/web/HomeController.java
- src/main/java/com/dozernet/common/web/JobCategory.java
- src/main/java/com/dozernet/common/web/NotificationController.java

Shared frontend templates

- src/main/resources/templates/fragments/common.html
- src/main/resources/templates/auth/login.html
- src/main/resources/templates/error/generic.html
- src/main/resources/templates/notifications/list.html
- src/main/resources/templates/public/about.html
- src/main/resources/templates/public/how-it-works.html
- src/main/resources/templates/public/landing.html

Static assets (CSS / JS / images)

- src/main/resources/static/css/app.css
- src/main/resources/static/js/hero.js
- src/main/resources/static/js/job-uses.js
- src/main/resources/static/images/ (all files)

Config & project files (project root)

- pom.xml
- README.md
- MODULES.md
- .gitignore
- src/main/resources/application.properties
- src/main/resources/application-h2.properties
- src/main/resources/application-mysql.properties
- docs/DESIGN_PATTERNS.md

A4. Set up MySQL

1. Start MySQL, then create the database:

```
mysql -u root -p
CREATE DATABASE dozernet;
EXIT;
```

2. Open `src/main/resources/application-mysql.properties` and set `spring.datasource.username` / `spring.datasource.password` to your own MySQL login (defaults to `root` / `root123` — change the password to match yours).

3. That's it — the app creates every table automatically on first run (`spring.jpa.hibernate.ddl-auto=update`) and seeds four demo accounts the first time the `users` table is empty.

A5. Run the app

Quick check (no database setup needed) — uses an in-memory H2 database:

```
mvn spring-boot:run
```

With your real MySQL database:

```
mvn spring-boot:run -Dspring-boot.run.profiles=mysql
```

Then open `http://localhost:8080`. Run the tests with:

```
JAVA_HOME=$(/usr/libexec/java_home -v 21) mvn test # macOS, if your default JDK is newer
```

A6. Demo login accounts (password: Password123)

Role	Email
Admin	admin@dozernet.lk
Customer	customer@dozernet.lk
Owner	owner@dozernet.lk
Operator	operator@dozernet.lk

A7. Push your copy to your own GitHub repository

Each of you pushes to your **own** repository (not one shared repo). Create a new, empty repository on GitHub first (no README/license — keep it empty), then from your project folder:

```
cd dozernet
git init
git add .
git commit -m "DozerNet - <your module name> module"
git branch -M main
git remote add origin https://github.com/<your-username>/<your-repo-name>.git
git push -u origin main
```

Optional but recommended: also create a branch named after your module (see the table on page 1) so your work is easy to find — `git checkout -b feature/mN-your-module && git push -u origin feature/mN-your-module`.

PART B — Your module: #1 Customer Management

G. Ananian · IT25102642

Package: `com.dozernet.module1_customer` | Suggested branch: `feature/m1-customer-management`

Your backend files (Java)

Copy these into `src/main/java/com/dozernet/module1_customer/`, keeping the same sub-folder each file is listed under below.

- `dto/ProfileForm.java`
- `dto/RegisterForm.java`
- `service/CustomerNotificationService.java`
- `service/CustomerService.java`
- `web/CustomerController.java`
- `web/CustomerNotificationController.java`
- `web/RegistrationController.java`

Your frontend files (Thymeleaf templates)

Copy these into `src/main/resources/templates/`:

- `customer/dashboard.html`
- `customer/notifications.html`
- `customer/profile.html`
- `customer/register.html`

Remember: your copy of the project also needs every file from **Part A, section A3** (the shared `common/` package, shared templates, static assets and config files) to actually compile and run — those are the same for all six of you and you don't need to modify them.

Viva tip: you should be able to explain every file above line by line — what business rule it enforces, which repository/service it talks to, and why. That's the part that's graded as your individual contribution.

PART B — Your module: #2 Booking & Rental Management

Theekshana A.D. · IT25101810

Package: `com.dozernet.module2_booking` | Suggested branch: `feature/m2-booking-rental`

Your backend files (Java)

Copy these into `src/main/java/com/dozernet/module2_booking/`, keeping the same sub-folder each file is listed under below.

- `CancellationPolicy.java`
- `SriLankaDistricts.java`
- `dto/BookingForm.java`
- `dto/ReturnInspectionForm.java`
- `entity/Booking.java`
- `entity/BookingStatus.java`
- `entity/MachineReplacementLog.java`
- `entity/ResponsibleParty.java`
- `entity/ReturnInspection.java`
- `repository/BookingRepository.java`
- `repository/MachineReplacementLogRepository.java`
- `repository/ReturnInspectionRepository.java`
- `service/BookingService.java`
- `service/ReturnInspectionService.java`
- `web/AdminBookingController.java`
- `web/CustomerBookingController.java`

Your frontend files (Thymeleaf templates)

Copy these into `src/main/resources/templates/`:

- `booking/admin-bookings.html`
- `booking/admin-inspections.html`
- `booking/admin-replace-machine.html`
- `booking/admin-return-inspection.html`
- `booking/my-bookings.html`
- `booking/new.html`

Remember: your copy of the project also needs every file from **Part A, section A3** (the shared `common/` package, shared templates, static assets and config files) to actually compile and run — those are the same for all six of you and you don't need to modify them.

Viva tip: you should be able to explain every file above line by line — what business rule it enforces, which repository/service it talks to, and why. That's the part that's graded as your individual contribution.

PART B — Your module: #3 Fleet (JCB) Management

Jayawardhana A.G.D.L. · IT25100798

Package: `com.dozernet.module3_fleet` | Suggested branch: `feature/m3-fleet-management`

Your backend files (Java)

Copy these into `src/main/java/com/dozernet/module3_fleet/`, keeping the same sub-folder each file is listed under below.

- `dto/MachineForm.java`
- `dto/OwnerRegisterForm.java`
- `entity/Machine.java`
- `entity/MachineStatus.java`
- `entity/MachineType.java`
- `entity/Ownership.java`
- `repository/MachineRepository.java`
- `service/FleetService.java`
- `web/AdminFleetController.java`
- `web/MachineCatalogController.java`
- `web/OwnerController.java`

Your frontend files (Thymeleaf templates)

Copy these into `src/main/resources/templates/`:

- `fleet/admin-listings.html`
- `fleet/admin-machine-form.html`
- `fleet/admin-machines.html`
- `fleet/catalog.html`
- `fleet/detail.html`
- `fleet/owner-dashboard.html`
- `fleet/owner-machine-form.html`
- `fleet/owner-register.html`

Remember: your copy of the project also needs every file from **Part A, section A3** (the shared `common/` package, shared templates, static assets and config files) to actually compile and run — those are the same for all six of you and you don't need to modify them.

Viva tip: you should be able to explain every file above line by line — what business rule it enforces, which repository/service it talks to, and why. That's the part that's graded as your individual contribution.

PART B — Your module: #4 Operator Management

Perera M.A.S.A. · IT25103606

Package: `com.dozernet.module4_operator` | Suggested branch: `feature/m4-operator-management`

Your backend files (Java)

Copy these into `src/main/java/com/dozernet/module4_operator/`, keeping the same sub-folder each file is listed under below.

- `dto/CompanyOperatorForm.java`
- `dto/DriverRegisterForm.java`
- `entity/Assignment.java`
- `entity/JobStatus.java`
- `entity/OperatorProfile.java`
- `repository/AssignmentRepository.java`
- `repository/OperatorProfileRepository.java`
- `service/OperatorService.java`
- `web/AdminOperatorController.java`
- `web/DriverRegistrationController.java`
- `web/OperatorController.java`

Your frontend files (Thymeleaf templates)

Copy these into `src/main/resources/templates/`:

- `operator/admin-assignments.html`
- `operator/admin-operators.html`
- `operator/dashboard.html`
- `operator/driver-register.html`

Remember: your copy of the project also needs every file from **Part A, section A3** (the shared `common/` package, shared templates, static assets and config files) to actually compile and run — those are the same for all six of you and you don't need to modify them.

Viva tip: you should be able to explain every file above line by line — what business rule it enforces, which repository/service it talks to, and why. That's the part that's graded as your individual contribution.

PART B — Your module: #5 Maintenance & Service Management

Ruvendi R.G.K. · IT25101701

Package: `com.dozernet.module5_maintenance` | Suggested branch: `feature/m5-maintenance-service`

Your backend files (Java)

Copy these into `src/main/java/com/dozernet/module5_maintenance/`, keeping the same sub-folder each file is listed under below.

- `dto/ScheduleForm.java`
- `entity/MaintenanceRecord.java`
- `entity/MaintenanceStatus.java`
- `repository/MaintenanceRecordRepository.java`
- `service/MaintenanceService.java`
- `web/AdminMaintenanceController.java`

Your frontend files (Thymeleaf templates)

Copy these into `src/main/resources/templates/`:

- `maintenance/admin-maintenance.html`
- `maintenance/history.html`

Remember: your copy of the project also needs every file from **Part A, section A3** (the shared `common/` package, shared templates, static assets and config files) to actually compile and run — those are the same for all six of you and you don't need to modify them.

Viva tip: you should be able to explain every file above line by line — what business rule it enforces, which repository/service it talks to, and why. That's the part that's graded as your individual contribution.

PART B — Your module: #6 Payment & Administration

Thathsara J.P.T. · IT25103683

Package: `com.dozernet.module6_payment` | Suggested branch: `feature/m6-payment-admin`

Your backend files (Java)

Copy these into `src/main/java/com/dozernet/module6_payment/`, keeping the same sub-folder each file is listed under below.

- `entity/Invoice.java`
- `entity/InvoiceStatus.java`
- `entity/Payment.java`
- `entity/PaymentMethod.java`
- `pricing/PricingBreakdown.java`
- `pricing/PricingSelector.java`
- `pricing/PricingStrategy.java`
- `pricing/StandardPricingStrategy.java`
- `pricing/TransportSurchargeCalculator.java`
- `report/ReportExporter.java`
- `report/ReportService.java`
- `report/ReportTable.java`
- `repository/InvoiceRepository.java`
- `repository/PaymentRepository.java`
- `service/AdminService.java`
- `service/CardPaymentRequest.java`
- `service/PaymentService.java`
- `web/AdminAuditController.java`
- `web/AdminDashboardController.java`
- `web/AdminInvoiceController.java`
- `web/AdminReportController.java`
- `web/AdminUserController.java`
- `web/CustomerInvoiceController.java`

Your frontend files (Thymeleaf templates)

Copy these into `src/main/resources/templates/`:

- `admin/dashboard.html` (note the folder: `templates/admin/`, not `templates/payment/`)
- `payment/admin-audit.html`
- `payment/admin-invoice-detail.html`
- `payment/admin-invoices.html`
- `payment/admin-reports.html`
- `payment/admin-users.html`
- `payment/customer-invoice-detail.html`
- `payment/customer-invoices.html`
- `payment/customer-pay.html`
- `payment/customer-thank-you.html`

Remember: your copy of the project also needs every file from **Part A, section A3** (the shared common/ package, shared templates, static assets and config files) to actually compile and run — those are the same for all six of you and you don't need to modify them.

Viva tip: you should be able to explain every file above line by line — what business rule it enforces, which repository/service it talks to, and why. That's the part that's graded as your individual contribution.